

Technical Lead – Practical Assessment

Submission Document — Andres Velandia

Submission Structure

```
tl_practical_assessment/  
  section_a/  
    architecture_explanation_Andres_Velandia.md  
    architecture_explanation_Andres_Velandia.pdf  
    architecture-diagram_Andres_Velandia.png  
    roadmap_Andres_Velandia.md  
    roadmap_Andres_Velandia.pdf  
  section_b_c/  
    src/  
    tests/  
    legacy/          ← previous iteration kept for reference  
    DECISIONS.md  
    SPEC.md  
    README.md  
    jest.config.ts  
    package.json  
    tsconfig.json  
  section_d/  
    section_d.md  
    section_d.pdf  
  prompts/          ← AI usage disclosure (see note below)
```

Written documents are authored in Markdown as the primary format.
PDF versions are included for easier reading where available.

Note on AI Usage

AI assistance (Claude) was used throughout this assessment and is disclosed as instructed. The `prompts/` folder documents every prompt used during the process — spec design, implementation, test suite, refactoring, and documentation — so the evaluator can follow exactly how AI was applied and which decisions were made independently by the candidate.

Section A – Architecture & Roadmap

See `section_a/`.

- **architecture_explanation_Andres_Velandia.md** — end-to-end target architecture for a multi-country Digital Direct Channel covering high availability, scalability, resilience, observability, and integration patterns (retries, circuit breaker, idempotency, bulkheads, async messaging, caching).
- **architecture-diagram_Andres_Velandia.png** — architecture diagram.
- **roadmap_Andres_Velandia.md** — 12-week technical roadmap with workstreams for Reliability, Integration Modernization, and Observability/Operations.

PDF versions of both written documents are included alongside the `.md` files.

Section B – Reusable Integration Framework

`resilience-client` is a transport-agnostic Node.js/TypeScript module in `section_b_c/`. It wraps any async function with the full resilience stack required by this section.

Quick start:

```
cd section_b_c
npm install
npm test                # 26 tests, all passing
```

Mechanisms implemented:

Mechanism	Approach
Timeout	<code>AbortController</code> + <code>Promise.race</code> per attempt
Retry + backoff	<code>min(baseDelay * 2^attempt, maxDelay)</code>
Jitter	<code>full</code> / <code>equal</code> / <code>none</code> — configurable
Circuit breaker	<code>CLOSED</code> → <code>OPEN</code> → <code>HALF_OPEN</code> state machine
Centralized config	Single <code>ResilienceConfig</code> passed to <code>createClient()</code>
Structured logging	JSON entries: <code>traceId</code> , <code>spanId</code> , <code>event</code> , <code>durationMs</code>
Trace propagation	W3C Trace Context-compatible IDs, OTel-ready
Idempotency	<code>IdempotencyStore</code> interface — in-memory default, Redis-swappable

See `section_b_c/SPEC.md` for behavioral contracts and `section_b_c/DECISIONS.md` for architectural rationale.

Section C – Demo Service & Reliability Test

The demo is `section_b_c/src/demo.ts`. It drives the integration framework directly against a simulated flaky upstream service and renders all failure and recovery events in real time.

Run it:

```
cd section_b_c
npx ts-node src/demo.ts          # interactive - pauses between scenarios
npx ts-node src/demo.ts --auto   # non-interactive, suitable for CI
```

4 scenarios:

Scenario 1 — Retry + Exponential Backoff + Jitter Upstream fails the first 3 calls, recovers on attempt 4. Each retry delay is computed with `equal` jitter and printed inline. Without retry the call fails permanently on attempt 1.

Scenario 2 — Circuit Breaker: CLOSED → OPEN → HALF_OPEN → CLOSED Upstream goes completely down. After 3 failures the circuit opens — subsequent requests are rejected instantly via `CircuitOpenError` without calling `fn()`. After `openTimeoutMs` the circuit probes and self-heals. State transitions print as badges as they happen.

Scenario 3 — Structured Logging + Level Filtering The same failing call runs twice: with `level: 'debug'` (all events visible) and with `level: 'warn'` (only failures surface). Demonstrates JSON output ready for any aggregator (Datadog, CloudWatch, ELK) without additional parsing.

Scenario 4 — All Patterns Combined A realistic payment service under stress, 4 parts in sequence: idempotency (3 concurrent duplicate requests → 1 `fn()` execution), retry (degraded gateway recovers on attempt 3), timeout (3s response cut at 900ms with `AbortSignal` cancellation), and circuit breaker (full outage → self-healing).

Expected final output:

Retry + Backoff + Jitter	recovered on attempt 4
Circuit Breaker	CLOSED→OPEN→HALF_OPEN→CLOSED, 2 requests protected
Structured Logging	JSON lines visible · level filter: 5 events → 1 at 'warn'
All patterns combined	idempotency + retry + timeout + circuit breaker all active

Section D – Technical Decision Record

See `section_d/`. Available as `.md` and `.pdf`.

Covers two decisions: centralized vs. decentralized integration platform, and event-driven vs. synchronous request-response architecture for critical flows. Each decision follows the format: context, options, decision, and consequences.

